Experiment 06: Docker Images and Deploy Containerized Applications using Docker

Experiment No.	06
Mapped CO	CO4
Aim	To create Docker images and deploy containerized applications using Docker.
Tools / Software	Docker Engine 24 or later; Ubuntu 22.04+ / Windows 11 with WSL2; Docker Hub account; A sample web application

Theory

In DevOps, application deployment should be repeatable, portable and easy to verify. Docker supports this requirement by packaging an application, runtime, dependencies and configuration into a deployable Docker image. The same image can be executed as a container on any compatible host.

A Docker image is a versioned, read-only package created from a Dockerfile. A container is the running instance of that image. For deployment, images are usually tagged with version names, pushed to a registry such as Docker Hub, pulled on a target machine and executed using docker run with suitable port mapping and restart options.

Common Dockerfile instructions such as FROM, WORKDIR, COPY, RUN, EXPOSE and CMD define how the image is built and how the application starts. During deployment, options such as -d, --name, -p and --restart help run the application in background mode, expose it to the host and keep it available after service restarts.

Procedure

1. Install Docker Engine and verify the installation using docker --version and docker info.
2. Create a project directory named docker-deployment-demo and place the sample application files inside it.
3. Create app.py with a simple Flask web route and create requirements.txt with the required dependency.
4. Write a Dockerfile defining the base image, working directory, dependency installation, exposed port and startup command.
5. Build a versioned Docker image using docker build and tag it as mywebapp:1.0.
6. Run the image locally as a container with port mapping and verify the output in a browser or using curl.
7. Tag the image for Docker Hub or registry-based deployment and optionally push it to the registry.
8. Deploy the image as a detached container using docker run with --name, -p and --restart options.
9. Verify deployment using docker ps, docker logs and browser/curl output.
10. Modify the application, build a new version such as mywebapp:2.0 and redeploy by replacing the old container.
11. Stop and remove containers/images after completion to clean up resources.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello from Docker deployed application!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, deploy, verify and redeploy commands

```
docker --version
docker build -t mywebapp:1.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:1.0
docker ps
curl http://localhost:8080
docker logs mywebapp
docker tag mywebapp:1.0 <dockerhub-user>/mywebapp:1.0
docker push <dockerhub-user>/mywebapp:1.0
docker stop mywebapp && docker rm mywebapp
docker build -t mywebapp:2.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:2.0
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker image creation, container deployment and verification. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date

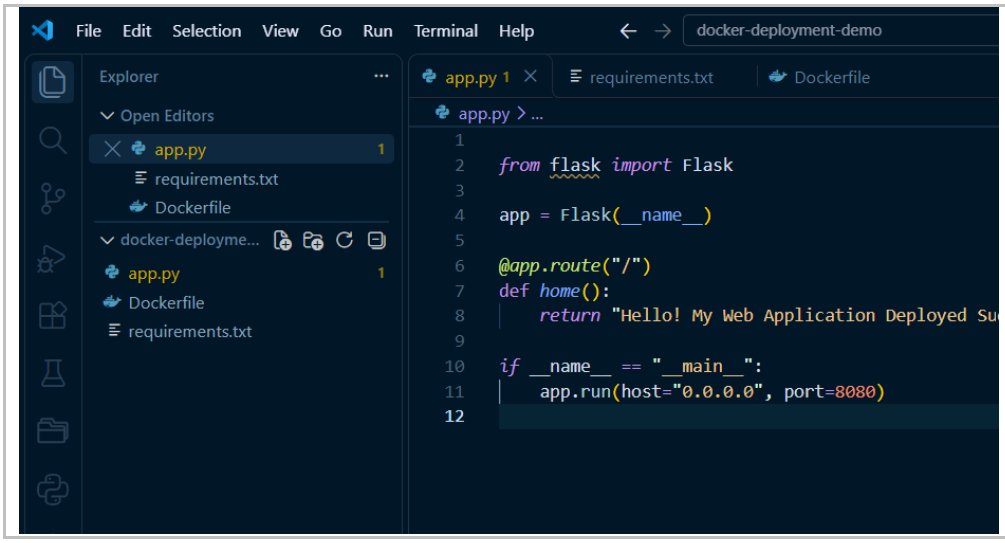
1. Docker Installation and Project Setup Output

Paste screenshot/output showing Docker version, Docker info and the created project directory.

```
C:\Users\Vijay>docker --version
Docker version 29.8.2, build 7fc2dff
```

2. Application Source and Dockerfile Output

Paste screenshot/output showing app.py, requirements.txt and Dockerfile contents.



3. Docker Image Build and Tag Output

Paste screenshot/output showing successful docker build command and image tag mywebapp:1.0.

```
C:\Users\Vijay\docker-deployment-demo>docker build -t mywebapp:1.0 .
[+] Building 2.4s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.1s
 => => transferring dockerfile: 221B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                 2.1s
 => [auth] library/python:pull token for registry-1.docker.io                                                       0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9   0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9   0.0s
 => [internal] load build context                                                                                   0.1s
 => => transferring context: 48B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => ERROR [5/5] COPY app.py .                                                                                       0.0s
------
 > [5/5] COPY app.py .:
------
Dockerfile:10
```

4. Container Deployment and Port Mapping Output

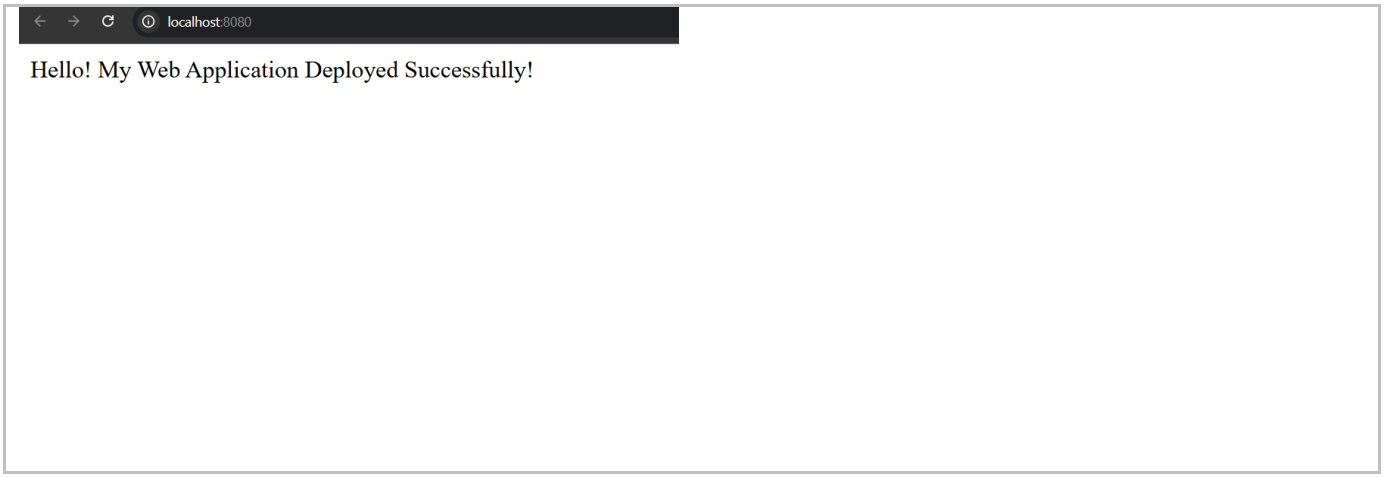
Paste screenshot/output showing docker run command, container name, restart policy and port mapping.

```
C:\Users\Vijay\docker-deployment-demo>docker run -d --name webapp-test --restart unless-stopped -p 8080:5000 mywebapp:2
0
c8614b61b787f419010ef3ce1b0bc0e1ad4c1f306c5ef445f7bbfcb5b7e7447c

C:\Users\Vijay\docker-deployment-demo>docker ps
CONTAINER ID   IMAGE           COMMAND           CREATED         STATUS         PORTS
     NAMES
c8614b61b787   mywebapp:2.0    "python app.py"   8 seconds ago   Up 8 seconds   0.0.0.0:8080->5000/tcp, [::]:8080->5000/
cp   webapp-test
```

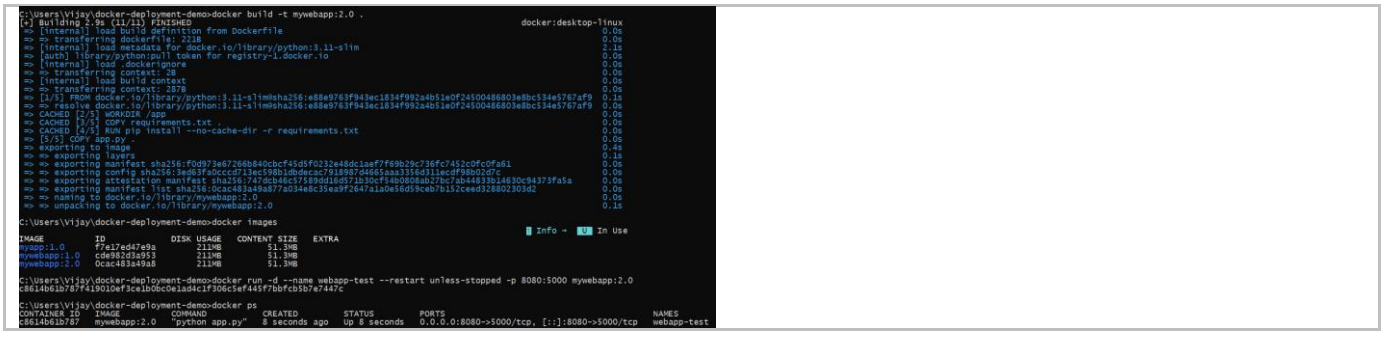
5. Application Access and Logs Output

Paste screenshot/output showing browser/curl response, docker ps and docker logs output.



6. Docker Hub Push / Pull or Redeployment Output

Paste screenshot/output showing image push/pull or updated version deployment such as mywebapp:2.0.



GitHub Repository / Lab Folder Link: https://github.com/nikamkrishna055-coder/EXP6

Docker Hub Repository Link (Optional):

Final Observation Statement:

```
Write 2-3 lines summarizing the result of the experiment.
```

Expected Output

Docker should be installed and working. The sample web application and Dockerfile should be created successfully. A versioned Docker image should be built, deployed as a container with port mapping, and verified through browser/curl output. Container status and logs should be visible using docker ps and docker logs. Optional registry push/pull or updated redeployment should also be demonstrated.

Conclusion

Docker images were created and used to deploy a containerized web application. The experiment demonstrated image building, tagging, container deployment, port mapping, log inspection and redeployment of an updated version.

Viva / Review Questions

1. What is the difference between a Docker image and a Docker container?
2. Why are version tags useful during Docker-based deployment?
3. Explain the role of port mapping in docker run.
4. What is the purpose of the --restart option during container deployment?

Marks: ____ / 10 **Date: _____** **Signature of Faculty: _____**